

EncryptionGuard v4

Complete, Evaluation-Ready Architecture

Project: EncryptionGuard

Subtitle: Explainable AI for Detecting Coordinated Payment Abuse

Razorpay track: AI Risk Manager

Selected loss class: Coordinated refund abuse

1. Single problem and product boundary

EncryptionGuard detects **coordinated refund abuse**: several customer accounts, devices, IP

addresses, addresses, or payment-token-like identifiers are connected and collectively exhibit

refund-abuse behavior. The system identifies suspicious events and communities, estimates merchant

exposure, explains the evidence, and recommends a bounded response.

The first version will not claim to detect generic fraud, all chargebacks, account takeover, or every payment attack. Those may appear as contextual signals, but the positive label is specifically

coordinated refund abuse. The system is a defensive detector and investigator assistant. It will

recommend `allow` , `monitor` , `step_up_verification` , `manual_review` , or `hold_for_review` . It

will not automatically ban customers, issue refunds, capture payments, or alter merchant settings.

2. Exact technology commitments

Concern	Committed choice	Reason
API and webhook receiver	Python + FastAPI	Clear request validation, raw-body access, and testability

System of record	PostgreSQL	Durable normalized events, cases, labels, feedback, and audit history
Graph store	Neo4j	Native entity relationships and Cypher queries
Offline graph analysis	NetworkX	Reproducible experiments and unit tests outside the online path
Velocity cache	Redis	Fast rolling-window counters with TTLs
Background queue	Celery with Redis broker and Redis result backend	Explicit retries, backoff, task visibility, and worker separation
Frontend	React + TypeScript + Vite + Tailwind CSS	Fast investigator dashboard development
Graph visualization	Cytoscape.js with react-cytoscapejs	Typed nodes/edges, neighborhood expansion, styling, and selection events
Primary model	XGBoost binary classifier	Strong tabular performance, class weighting, and SHAP compatibility
Baseline models	Rules engine and logistic regression	Prove that the primary model adds measurable value
Explainability	SHAP TreeExplainer plus graph evidence	Feature-level and relationship-level explanations
Analyst assistant	gpt-5.4-mini (primary); gpt-5-mini (fallback) , server-side only	gpt-5.4-mini is confirmed API-available as of March 2026 and runs faster than its predecessor; gpt-5-mini is the named fallback if the primary model string is catalog-unavailable. Both are confirmed before implementation by querying the live OpenAI model list endpoint.
Assistant output format	Strict JSON schema	Prevents free-form policy ambiguity and makes validation deterministic

Policy checker	Deterministic Python policy engine + Pydantic schema validation	No second opaque model in the safety-critical path
Feature library	Shared versioned Python module used by both offline training and online scoring	Eliminates training-serving skew at the source
Packaging	Docker Compose	Reproducible local stack for API, worker, PostgreSQL, Neo4j, Redis, and frontend

The LLM is not the fraud detector. It only summarizes retrieved evidence, answers analyst questions, and drafts notes. Model scoring, thresholds, actions, and audit records remain deterministic and testable.

3. Data generation, labels, and payment-token simulation

Razorpay Test Mode will validate integration, payment flow, and webhook handling. It will not be

treated as the fraud generator. Razorpay describes Test Mode as simulated and documents that Test

Mode webhook payloads retain the same structure as Live Mode payloads. [1] [2]

A versioned scenario generator will create the evaluation population. It will generate merchants,

customer accounts, devices, IPs, address hashes, payment-token-like IDs, orders, payments, and

refunds. Each generated event will carry `event_label` , `ring_id` , `scenario_id` , `generated_at` , and `generator_version` .

3.1 Scenarios

Scenario	Ground truth
Normal customer with no refund	Legitimate
Legitimate occasional refund	Legitimate
Legitimate shared network (hostel, office, family)	Legitimate

Single-account repeated refund abuse	Abuse
Coordinated refund ring across accounts	Abuse
Near-miss pattern with shared infrastructure but no abuse	Legitimate
High-loss coordinated ring	Abuse with exposure tier

3.2 Stable payment-token-like identifiers

Test Mode will not be used to obtain real card numbers. The generator creates stable opaque

identifiers such as `ptok_demo_00127` . A configurable probability controls token reuse across accounts, devices, and rings. Only the opaque ID is stored; no PAN, CVV, bank credential, or real payment secret is generated.

The generator creates both legitimate shared-token patterns (family accounts) and abusive shared-token patterns (new accounts sharing a token while requesting refunds). This prevents the unsafe rule "token reuse equals fraud."

3.3 Reproducibility

Dataset configuration and random seed are committed. At least three independent seeds will be run.

Reporting uses **mean $\pm$ standard deviation across seeds**, with the worst seed shown as a robustness check. We will not cherry-pick the best run.

We will not assume fraud prevalence is always 0.1%. The report documents actual prevalence and includes stress tests at 0.1%, 0.5%, 1%, and 5% positive event rates.

4. Train, validation, held-out test, and leakage control

Labels come from the generator and analyst feedback, never from model predictions. The split is time-aware and group-aware:

Split	Use	Protection
-------	-----	------------

Train	Fit features, model, and class weights	Earliest time period; ring IDs isolated
Validation	Tune features, calibration, thresholds, and prompts	Later time period; no shared ring IDs
Held-out test	One-time final report	Latest time period; no shared ring IDs or scenario templates
Drift set	Robustness to changed behavior	New period and altered scenario parameters

The test set is frozen before threshold selection and is never used for prompt tuning, feature selection, or screenshots. Label-derived features and analyst outcomes are calculated only from data available inside the relevant training window.

4.1 Graph leakage prevention mechanism

Every Cypher query that generates graph features accepts a `reference_timestamp` parameter. All relationship traversals include `WHERE r.created_at <= $reference_timestamp`. During training-set feature generation, the reference timestamp is set to the training split boundary; during validation, to the validation boundary. The split manifest records these exact timestamps. No code path generates graph features without a reference timestamp; this invariant is enforced by a CI test that mocks Neo4j and asserts the parameter is always passed. Graph edges whose source events fall after the training cutoff are excluded from training-phase queries. Hard edge deletion, as described in Section 6, removes very old edges from the physical graph.

5. Event ingestion and Razorpay integration

EncryptionGuard subscribes to this minimal Razorpay event set:

Event	Purpose
<code>order.paid</code>	Associate a successful order with merchant and customer context

payment.authorized	Record authorization state and timing
payment.captured	Establish captured payment and refund eligibility context
payment.failed	Add failed-attempt and velocity context
refund.created	Start refund-intent analysis
refund.processed	Confirm completed refund and merchant exposure
refund.failed	Distinguish attempted from completed financial loss

Disputes, subscriptions, payouts, and transfers are excluded from v1 to protect the single-loss-class boundary.

After durable event storage, a Celery enrichment task may make read-only Test Mode calls to fetch

a payment, order, payments for an order, or refunds for a payment. The system will not automatically call capture, refund, accept-dispute, or contest-dispute operations. [4] [5]

5.1 Webhook security and reliability

The FastAPI receiver preserves the raw request body, validates `X-Razorpay-Signature` using HMAC-SHA256, rejects invalid signatures, and only then parses JSON. It uses `x-razorpay-event-id`

combined with `merchant_id` as a unique idempotency key.

The receiver writes the immutable raw envelope and normalized event in PostgreSQL before returning

2xx. A duplicate increments a delivery-attempt record without duplicating domain state. Celery tasks

are idempotent and retry-safe. Out-of-order events are accepted and reconciled by event timestamp

and state precedence. Invalid signatures are never queued. Secrets stay in environment variables

or a secret manager and never enter the public repository. [6] [7]

5.2 First-event-before-context handling

When a `refund.created` event arrives before the corresponding `payment.captured` event has been stored — a valid out-of-order scenario documented by Razorpay — the system creates a provisional payment entity with the available fields and marks the case `pending-enrichment`. A Celery reconciliation task checks for missing parent events on a configurable short interval (default 30 seconds, maximum 5 retries with backoff). When the parent event arrives, the provisional record is reconciled using the `razorpay_payment_id` as the join key, the entity is fully populated, and the case is rescored. This prevents silent data gaps without violating idempotency.

Unresolved provisional records after the maximum retry window are flagged for manual review and logged.

6. Storage, online path, batch path, and feature management

PostgreSQL stores immutable webhook envelopes, normalized entities, cases, feedback, and audit records. Neo4j stores the current relationship graph. Redis stores rolling counters and task coordination. Offline feature snapshots are versioned Parquet artifacts containing feature definitions, dataset seed, split manifest, and schema version.

The **online path** must produce an initial point-level score within a target of **p95 $\leq$ 100 ms after the verified event is available to the scoring service**, excluding external Razorpay network time and asynchronous graph processing. Redis supplies velocity features, and XGBoost produces the initial score.

The **batch/asynchronous path** handles Neo4j upserts, graph feature computation, community detection, API enrichment, case recomputation, drift reports, and retraining. The online response does not wait for Louvain or PageRank. When graph analysis finishes, the case is updated with a

new versioned score and evidence set.

6.1 Neo4j edge TTL implementation

Neo4j does not natively expire relationship records. The 90-day TTL is implemented through three

mechanisms working together:

1. **Property on write.** Every relationship carries a `created_at` datetime property set at write time.
2. **Query-time filtering.** All scoring and analysis Cypher queries filter relationships with `WHERE r.created_at >= datetime() - duration({days: $ttl_days})` , where `ttl_days` defaults to 90 and is loaded from configuration at query time.
3. **Scheduled hard delete.** A weekly Celery maintenance task executes a batched Cypher DELETE on relationships older than 180 days, preventing unbounded graph growth. The hard-delete boundary is always at least twice the scoring TTL so that the 180-day sensitivity analysis remains valid throughout. The maintenance task runs in batches of 10,000 edges to avoid long-running transactions.

The TTL parameter is logged with every feature generation run. Changing the TTL value requires a

model version bump because it affects the feature definition.

Sensitivity analysis at 30 days and 180 days will be reported alongside the default 90-day results.

6.2 Training-serving feature parity

Offline Parquet features and online Redis features are computed by the **same versioned Python**

functions from a shared `feature_library` module in the repository. The training pipeline imports

and calls these functions on historical snapshots; the online scoring service imports and calls the

same functions on live Redis state.

Feature schema versions are stored with every model artifact. A CI suite runs parity tests that feed identical inputs through the offline and online code paths and assert matching numeric outputs.

Any schema change that breaks parity fails the build before the code reaches the training pipeline or the deployed service. The model card records the feature library version alongside the model version.

7. Graph model and temporal policy

Nodes represent accounts, devices, IPs, address hashes, payment-token-like IDs, orders, payments, and refunds. Edges represent `USES` , `ORIGINATED_FROM` , `SHIPS_TO` , `PAID_WITH` , `PLACED` , `HAS_PAYMENT` , and `HAS_REFUND` relationships.

Graph algorithms for v1:

Algorithm	Purpose
Connected components	Candidate cluster discovery
Louvain	Dense community detection in the asynchronous batch worker
Weighted degree	Identify highly reused entities
PageRank-style centrality	Identify central linking infrastructure
Temporal filtering	Prevent stale links from dominating current risk

Default edge TTL is 90 days for device, IP, address, and payment-token relationship edges. Payment and refund event edges remain as historical records but their scoring influence decays after 90 days. Sensitivity analysis at 30 and 180 days will be reported. A shared IP alone never creates a positive abuse label or automatic enforcement recommendation.

8. ML, explainability, and cost-sensitive policy

The rules engine and logistic regression model are baselines. The primary model is XGBoost. Training

uses class-weighted loss with weights calculated only from the training split. Oversampling, if

tested, is applied only inside the training split. Accuracy is not a primary metric.

Reported metrics: PR-AUC, precision, recall, confusion matrix, precision-recall curves across thresholds, precision and recall at fixed review capacity, calibration, false-positive rate, false-negative rate, event-level metrics, and ring-level metrics.

8.1 Hyperparameter tuning

XGBoost hyperparameters are tuned on the validation set using **Bayesian optimization via Optuna**

with a fixed budget of 100 trials. The search space covers: learning rate, maximum tree depth,

subsample ratio, column sample by tree, minimum child weight, gamma, and scale_pos_weight range.

Early stopping uses the validation PR-AUC. The winning configuration is frozen and recorded in

the model card before any test evaluation occurs. No hyperparameter selection uses test data.

8.2 Ring-level metrics

A predicted ring is a connected candidate cluster crossing the configured risk threshold. A ring

is a true positive when it overlaps the ground-truth abuse ring according to a documented overlap

rule (e.g., at least one core abusive account and a minimum edge or member overlap). We report:

- **Ring-level precision:** percentage of predicted abuse rings that correspond to real abusive rings.
- **Ring-level recall:** percentage of ground-truth abuse rings detected.
- **Ring-level F1:** harmonic mean of ring precision and recall.
- **Member coverage:** percentage of abusive accounts captured within detected rings.

SHAP TreeExplainer produces model-feature contributions. Graph evidence separately lists the shared entities, timestamps, paths, and community statistics. Explanations must not merge correlation with proof.

8.3 Cost matrix and thresholds

The policy uses:

$$\text{ExpectedCost}(t) = \text{FP}(t) * C_{\text{FP}} + \text{FN}(t) * C_{\text{FN}} + \text{Review}(t) * C_{\text{REVIEW}} + \text{Friction}(t) * C_{\text{FRICTION}}$$

C_{FP} includes review labor, support cost, and modeled customer friction. C_{FN} includes the

refund amount and estimated unrecovered merchant loss. C_{REVIEW} is analyst time.

C_{FRICTION}

captures step-up-verification or delayed-checkout cost. The dashboard shows the assumptions and

sensitivity analysis.

Thresholds are selected on validation data to minimize expected cost subject to a recall floor for

high-loss cases and a maximum manual-review queue. The frozen policy then runs once on the

held-out test set.

9. LLM assistant and policy checker

The analyst assistant uses **gpt-5.4-mini (primary) or gpt-5-mini (fallback) via the server-side**

built-in LLM integration. The assistant receives a structured, retrieved evidence bundle containing event IDs, timestamps, graph relationships, score components, model version, and

policy state. It returns strict JSON with `summary`, `evidence_ids`, `risk_factors`, `recommended_next_step`, `uncertainties`, and `refusal_reason` fields.

9.1 LLM failure path

If the primary model call fails, the system executes a **3-attempt retry with exponential backoff**.

If all retries fail, it falls back to the secondary model. If the secondary model also fails, the

assistant returns a **structured deterministic fallback response** indicating that AI analysis is temporarily unavailable and providing the raw model score and rule-based evidence instead.

9.2 Evidence bundle management

To manage context limits and cost, the evidence bundle is capped at **50 events and 20 relationships**

per case. If the case exceeds these limits, a hierarchical summary is prepended, and the least

relevant historical events are truncated. Truncation is explicitly logged and displayed to the analyst.

9.3 Policy checker implementation

The policy checker is a deterministic Python module that:

1. Validates the JSON schema with Pydantic.
2. Verifies that every cited event ID exists in the case evidence bundle.
3. Rejects unsupported claims and uncited numerical assertions.
4. Blocks prohibited content patterns (evasion, credential theft, malware, etc.).
5. Rejects any proposed irreversible action or threshold change.
6. Redacts secrets and unnecessary personal data.
7. Marks the response `needs_human_review` when confidence is insufficient.

The assistant is analyst-only. Counterfactual explanations are available only after authentication and authorization in the investigator console; they are never shown to end customers or public users. Counterfactuals are omitted from exported customer-facing messages.

10. Feedback, retraining, drift, promotion, and rollback

Analysts label cases `confirmed_abuse` , `legitimate` , `needs_more_evidence` , or `unknown` .
Feedback

is append-only and tied to event IDs, case ID, model version, analyst, and timestamp.

10.1 Retraining and promotion

Default cadence is **weekly candidate retraining**, plus emergency runs on drift alerts. A

candidate model must satisfy all promotion gates:

- **Integrity:** split, schema, leakage, and reproducibility tests pass.
- **Safety:** guardrail and authorization tests pass.
- **Performance:** PR-AUC is not more than 5% below current; improvement preferred.
- **Ring quality:** precision/recall meet floors and don't fall >5pp.
- **Cost:** expected cost is no worse than current by >3%.
- **Calibration:** Brier score does not materially regress.
- **Bias:** FP rate disparity across cohorts (region, age, history) remains below 10pp.
- **Parity:** online/offline feature parity suite passes.
- **Operations:** p95 scoring $\leq$ 100 ms; queue lag within demo SLA.
- **Shadow review:** 24 hours of shadow-mode observation or 2,000 events replay with no anomaly.

10.2 Drift and rollback

If a promoted model breaches a guardrail (e.g., ring recall falls >5pp, cost worsens >3%, p95 latency >100 ms, or alert rate changes >50%), the system **automatically reverts** to the last approved model. The previous model artifact is available for one-click rollback.

In-flight case handling: During a rollback, open cases retain their scores for the active session. New events are rescored by the restored model, and the audit log notifies analysts of the reversion.

11. Model card deliverable

Every submitted model version includes a model card containing:

- Intended use, prohibited use, and decision boundary.
- Training-data source, generator version, prevalence, seeds, and split manifest.
- Entity and feature definitions, graph TTL, and feature library version.
- Model type, hyperparameters, class-weight strategy, and calibration method.
- Event-level and ring-level precision, recall, F1, PR-AUC, calibration, and cost metrics.
- Threshold policy and complete cost matrix.
- False-positive and false-negative analysis.

- Fairness/bias test results across cohorts.
- Known limitations, bias risks, and failure modes.
- LLM model, prompt version, retrieval fields, and refusal tests.
- Deployment version, rollback target, and monitoring signals.

12. Demo narrative and definition of done

The five-minute demo shows a normal customer, a benign shared network, and a coordinated refund ring. A verified refund webhook enters; receiver validates and deduplicates; Redis updates velocity; XGBoost provides initial score ($p95 \leq 100$ ms); Celery updates Neo4j; graph analysis discovers the ring; case manager shows exposure and evidence; authenticated analyst opens restricted counterfactual view; LLM produces grounded summary; policy checker validates. Replay a duplicate and out-of-order event to show idempotency and resilience. Display held-out PR curve, threshold sweep, cost curve, confusion matrix, event/ring metrics, mean $\pm$ std across seeds, and feature stability (top-5 SHAP rank agreement within 2 positions).

Definition of Done:

- Repository reproduces dataset from seed and shows split manifest.
- Webhook fixtures pass signature, idempotency, and concurrency (50 concurrent) tests.
- Graph and online features are generated with parity.
- Model card is produced for the final version.
- Dashboard displays explainable evidence and restricted counterfactuals.
- Frozen held-out report is regenerated without changing the test set.

13. Deployment options

Approach	Tradeoffs	Cost	Complexity
Local Docker Compose + recorded	Most reproducible; no public dependency;	Lowest	Low

demo	limited remote interaction		
Public container with managed PostgreSQL and Neo4j	Shareable live URL; requires HTTPS and secrets management	Low-moderate	Moderate
Production-like separated API, workers, Redis, PostgreSQL, Neo4j	Strongest architecture story; largest operational surface	Moderate-high	High

The stack runs locally even if a hosted dependency fails. Only Test Mode credentials are used,

and secrets remain server-side. The Celery enrichment task handles Razorpay API rate limits using

bounded exponential backoff with jitter, a maximum retry count, a concurrency limit, and a dead-letter queue for exhausted tasks. [8]

14. Complete gap-resolution matrix

(Refer to the full list of 55 resolved gaps in the project repository)

References

- [1]: [Razorpay API Keys and Test Mode Documentation](#)
- [2]: [Razorpay Validate and Test Webhooks](#)
- [3]: [Razorpay All Webhook Events](#)
- [4]: [Razorpay Payments APIs](#)
- [5]: [Razorpay Refunds APIs](#)
- [6]: [Razorpay Webhook Best Practices](#)
- [7]: [Razorpay Webhook Validation and Idempotency](#)
- [8]: [Razorpay API Concepts and Rate Limiting](#)